

Music Recommendation with Knowledge Graphs and GNNs

Deep Learning — Milestone 2 Addendum

Group 7: Pedro Fanica (54346)

Aarthi Perumpillichira (66404)

Maja Skwaroń (66083)

This addendum extends our Milestone-1 report with the training methods, experiments and results produced since the previous submission. The data, knowledge-graph construction and overall problem framing are unchanged; we refer to the Milestone-1 report for those details.

1. Training Methods

We train three representation/recommendation components on top of the music knowledge graph, and compare them against three baselines on a shared, stratified user-level interaction split.

Symbolic-feature autoencoder. The 1,526-dimensional jSymbolic descriptor vector per track is standardised and compressed by a fully connected autoencoder ($1526 \rightarrow 512 \rightarrow 256 \rightarrow 128$ with a symmetric decoder), trained for 30 epochs with Adam (lr 10^{-3} , batch 256) to minimise reconstruction MSE. The 128-d bottleneck is used as a dense *content* embedding for each track.

Knowledge-graph embeddings. We train RotatE and ComplEx (PyKEEN, 128-d complex / 256-d real, 100 epochs, Adam) on the enriched graph with listening events excluded from the training corpus, giving a structure-only entity representation.

Heterogeneous Graph Transformer (main model). Our principal recommender is an HGT over the typed graph (hidden 128, 4 heads, 2 layers, dropout 0.1; 100 epochs, Adam lr 10^{-3} , weight decay 10^{-4}), with track nodes initialised from the autoencoder embeddings. It is trained with a *popularity-debiased listwise* loss (log-popularity prior, $\lambda = 0.2$, temperature 0.1) and scores items by user-track inner product with seen items masked.

Baselines. MostPopular; user-user KNN-CF (cosine, k chosen by validation sweep); and a two-stage XGBoost hybrid (autoencoder-profile candidate generation followed by `rank :ndcg` re-ranking on `[user || track]` content features).

Table 1. Top-10 test performance (Cov. = coverage).

Model	Recall	NDCG	HitRate	Cov.	Overall
MostPopular	0.1285	0.1143	0.2002	0.0064	0.1477
KNN-CF	0.0923	0.0971	0.1540	0.9176	0.4185
XGB-Hybrid	[TODO:]				
HGT (ours)	[TODO:]				

2. Experiments

All models share one user-level stratified train/val/test split (stratified by genre and release decade); only interactions are partitioned, so song features and graph structure are visible to all splits and no leakage occurs. We report Recall@ K , NDCG@ K , Hit-Rate@ K , MRR, catalogue Coverage and Popularity-Bias@ K for $K \in \{5, 10, 20, 50, 100\}$, plus a composite Overall score $0.6 \text{NDCG} + 0.2 \text{Coverage} + 0.2(1 - \text{PopBias})$. Pair-wise Wilcoxon and an omnibus Friedman test assess significance. **[TODO: State the train/val/test interaction counts.]**

3. Results

Tab. 1 reports top-10 test performance. Popularity is accurate but nearly zero-coverage; KNN-CF wins the composite via far higher coverage and lower popularity bias. **[TODO: Fill the hybrid and HGT rows once the autoencoder embeddings are regenerated, and state whether the GNN improves the accuracy/diversity trade-off.]**

Learning curves. Fig. 1 shows the training diagnostics: autoencoder reconstruction MSE (train/val), KGE loss, the HGT debiased-listwise loss with validation Recall/NDCG, and the XGBoost per-round `ndcg@10`. **[TODO: Insert the PNGs exported to `data/final/`.]**

Significance & qualitative. Accuracy gaps are statistically significant ($p < 0.05$) but small in effect size ($d_z \approx 0.03\text{--}0.11$); the decisive differences are in coverage and popularity bias. Population- and user-level qualitative analyses (genre/tempo/decade distribution shift, per-user case



[TODO: Insert learning curves (AE / KGE / HGT / XGBoost).]

Figure 1. Learning curves for the trained models.

studies) are summarised in the main report. [TODO: Add one qualitative figure if space permits.]

4. Contribution of Each Member

[TODO: Confirm the split below.]

- **Pedro Fanica:** [TODO: autoencoder, HGT + debiased loss, evaluation/qualitative framework, integration.]
- **Aarthi Perumpillichira:** [TODO: data linking, jSymbolic features, KNN/XGBoost baselines.]
- **Maja Skwaroń:** [TODO: KG construction, Wiki-data/OWL enrichment, KGE baselines.]